In [ ]:
```
# GitHub repository: https://github.com/edjavi120/m10-activity
import marimo as mo
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score, root_mean_squared_error
from sklearn.model_selection import train_test_split
```

Lung Function

A respiratory clinic ran routine spirometry on 500 healthy adult men who have never smoked (a simulated dataset, so some patterns are stronger than they would be in real clinic data). Each row is one participant, with these columns:

- standing height, in centimetres
- age, in years
- forced expiratory volume in one second (FEV1), in litres, a measure of lung function where a higher value means more air is exhaled quickly

Lung Function Data

Instructions: Load `lung_function.csv` into a DataFrame and display it.

In [ ]:
```
# Load the dataset
df = pd.read_csv("lung_function.csv")
df
```

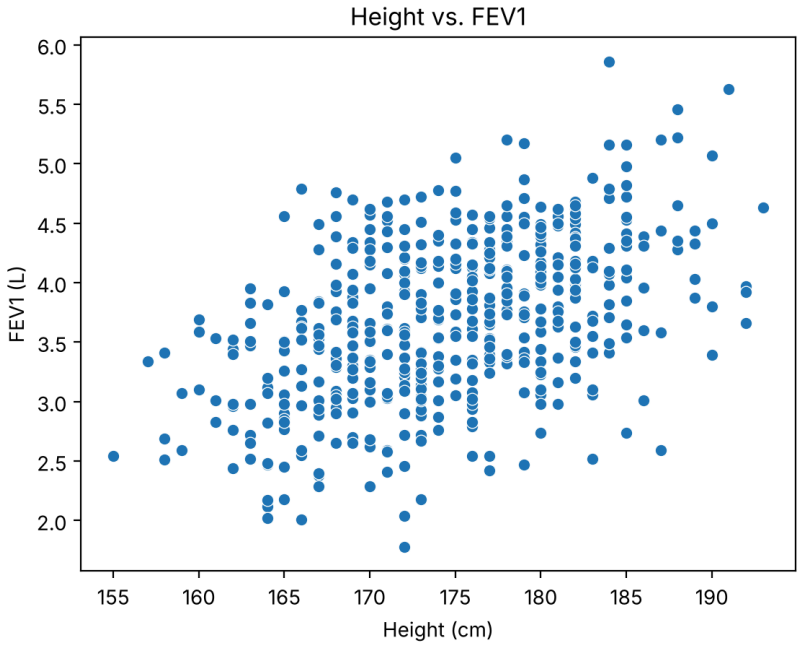
	height_cm	age_years	fev1_l
0	169	65	2.91
1	161	39	2.83
2	179	50	3.42
3	180	38	4.09
4	173	65	3.42
...	...	...	...
495	174	27	4.40
496	171	21	4.56
497	182	42	4.31
498	178	45	3.73
499	172	44	3.90

500 rows × 3 columns

Height vs. FEV1: Scatter Plot

Instructions: Create a scatter plot of height and FEV1.

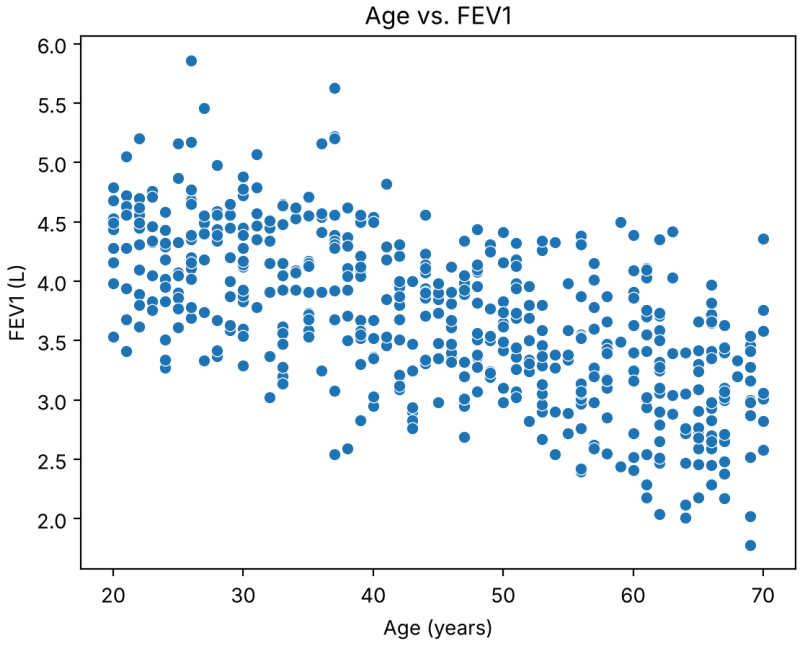
In [ ]:
```
# Scatter plot of height against FEV1
sns.scatterplot(df, x="height_cm", y="fev1_l")
plt.title("Height vs. FEV1")
plt.xlabel("Height (cm)")
plt.ylabel("FEV1 (L)")
plt.show()
```



Age vs. FEV1: Scatter Plot

Instructions: Create a scatter plot of age and FEV1.

In [ ]:
```
# Scatter plot of age against FEV1
sns.scatterplot(df, x="age_years", y="fev1_l")
plt.title("Age vs. FEV1")
plt.xlabel("Age (years)")
plt.ylabel("FEV1 (L)")
plt.show()
```



Height vs. FEV1: Correlation

Instructions: Calculate the correlation between height and FEV1.

In [ ]:
```
# Correlation of height with FEV1
df["height_cm"].corr(df["fev1_l"])
```

np.float64(0.4684818276405379)

Age vs. FEV1: Correlation

Instructions: Calculate the correlation between age and FEV1.

In [ ]:
```
# Correlation of age with FEV1
df["age_years"].corr(df["fev1_l"])
```

np.float64(-0.6324051508544049)

Predictors

Instructions: Store height and age as the predictors, `X`.

In [ ]:
```
# Predictors (X): height and age
X = df[["height_cm", "age_years"]]
```

Response

Instructions: Store FEV1 as the response, `y`.

In [ ]:
```
# Response (y): FEV1
y = df["fev1_l"]
```

Training and Test Sets

Instructions: Split `X` and `y` into training and test sets, with 20% of the rows in the test set. Name them `X_train`, `X_test`, `y_train` and `y_test`, and display the number of rows in the training and test sets.

In [ ]:
```
# Split into training and test sets, then show the number of rows in each
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
len(X_train), len(X_test)
```

```
(400, 100)
```

Regression Model

Instructions: Create a linear regression model named `model` and fit it to the training set. Display the slopes and the intercept of the fitted model.

In [ ]:
```
# Create the model and fit it to the training set, then show the slopes (height first, then age) and the interce
model = LinearRegression()
model.fit(X_train, y_train)
model.coef_, model.intercept_
```

```
(array([ 0.04550379, -0.02855901]), -2.9719259326131313)
```

Training Predictions

Instructions: Predict FEV1 for the training set and store the predictions as `train_pred`.

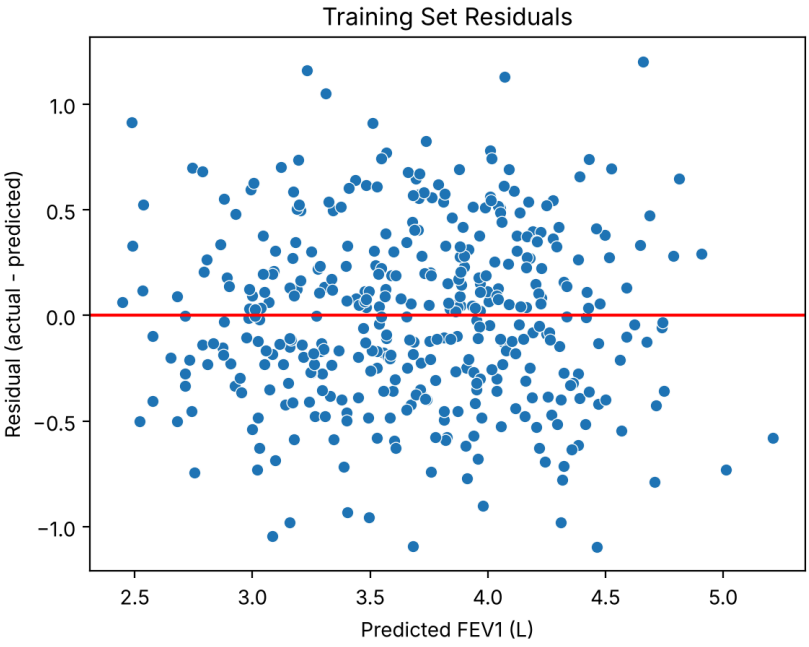
In [ ]:
```
# Predict FEV1 for the training set
train_pred = model.predict(X_train)
```

Residual Plot

Instructions: Create a scatter plot of the predicted FEV1 and the residual (actual minus predicted) for the training set.

In [ ]:
```
# Residual plot: actual minus predicted, against predicted
sns.scatterplot(x=train_pred, y=y_train - train_pred)
plt.axhline(0, color="red")
```

```
plt.title("Training Set Residuals")
plt.xlabel("Predicted FEV1 (L)")
plt.ylabel("Residual (actual - predicted)")
plt.show()
```



Test Predictions

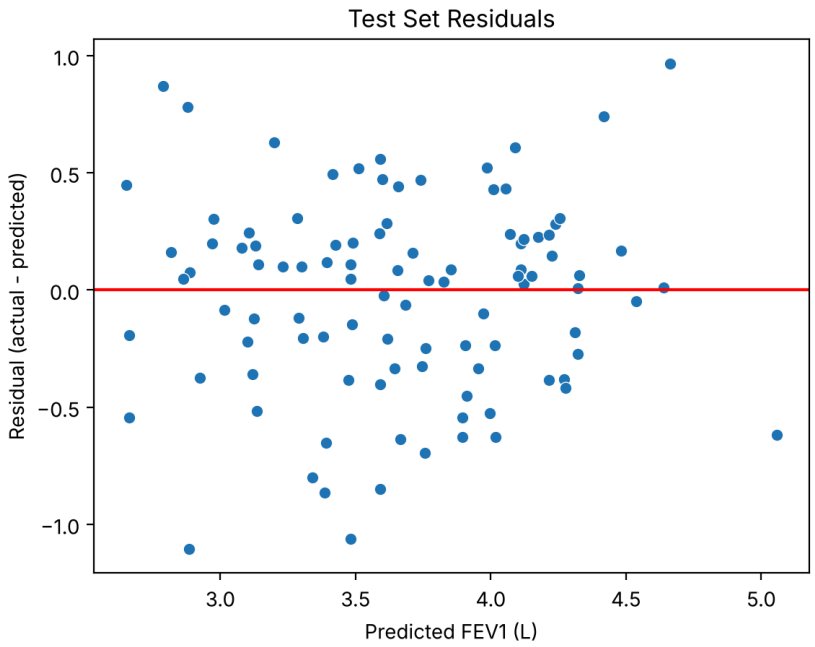
Instructions: Predict FEV1 for the test set and store the predictions as `y_pred`.

In [ ]:
```
# Predict FEV1 for the test set
y_pred = model.predict(X_test)
```

Test Score Residuals

Instructions: Create a scatter plot of the predicted FEV1 and the residual (actual minus predicted) for the test set.

In [ ]:
```
# Residual plot for the test set: actual minus predicted, against predicted
sns.scatterplot(x=y_pred, y=y_test - y_pred)
plt.axhline(0, color="red")
plt.title("Test Set Residuals")
plt.xlabel("Predicted FEV1 (L)")
plt.ylabel("Residual (actual - predicted)")
plt.show()
```



Test R Squared

Instructions: Calculate the R squared of the model on the test set.

In [ ]:
```
# Test R squared
r2_score(y_test, y_pred)
```

0.6113216781973615

Test MAE

Instructions: Calculate the MAE (mean absolute error) of the model on the test set.

In [ ]:
```
# Test MAE
mean_absolute_error(y_test, y_pred)
```

0.33070843916167475

Test RMSE

Instructions: Calculate the RMSE (root mean squared error) of the model on the test set.

In [ ]:
```
# Test RMSE
root_mean_squared_error(y_test, y_pred)
```

0.4166398774102865

Predict for New Patients

Instructions: Four new participants are every combination of two heights, 165 cm and 185 cm, and two ages, 30 and 60 years. Predict the FEV1 of each one.

In [ ]:
```
# Predict FEV1 for each new participant (every combination of height and age)
model.predict(
    pd.DataFrame({"height_cm": [165, 165, 185, 185], "age_years": [30, 60, 30, 60]})
)
```

array([3.67942915, 2.82265898, 4.58950493, 3.73273476])